

## **Trabajo Práctico N°II - Unidad 4**

### **Gestión Colaborativa, Control de Versiones y Organización Empresarial (Git, GitHub y Jira)**

#### **Alumnos - Comisión 21**

Agustina Balcarcel

**Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.**

#### **Organización Empresarial**

##### **Docente Titular**

Prof. Gabriela Martínez

##### **Docente Tutor**

Prof. Mario Raúl López, Prof. Andrea Ramos, Prof. Carolina Bruno

19 de junio de 2025

### **Trabajo Práctico Integrador**

#### **Introducción**

Deberán identificar un proceso administrativo crítico dentro de una organización (ej. gestión de vacaciones, soporte técnico nivel 1, alta de proveedores) y automatizarlo mediante un chatbot, asegurando que la lógica del bot responda a un modelo de procesos estandarizado. Realizarlo como Simulación de proceso

#### **Objetivos**

Deberán identificar un proceso administrativo crítico dentro de una organización (ej. gestión de vacaciones, soporte técnico nivel 1, alta de proveedores) y automatizarlo mediante un chatbot, asegurando que la lógica del bot responda a un modelo de procesos estandarizado. Realizarlo como Simulación de proceso.

#### **Consigna**

Deberán entregar un diagrama BPMN 2.0 de alta resolución que represente fielmente el flujo del bot. El modelo debe incluir:

- Carriles (Lanes): Diferenciar claramente las acciones del "Usuario" y las del "Sistema/Bot".
- Eventos: Definir hitos de inicio y fin del proceso.
- Compuertas (Gateways): El bot debe tomar al menos dos caminos lógicos distintos basados en decisiones (ej. ¿tiene saldo de días?, ¿monto aprobado?).

## Desarrollo

[https://github.com/BautistaMattei/TPI\\_ORG\\_EMP\\_BESI\\_MATTEI/blob/main/README.md](https://github.com/BautistaMattei/TPI_ORG_EMP_BESI_MATTEI/blob/main/README.md)

### Contexto organizacional

La organización elegida para este trabajo es "TeleConecta", una empresa ficticia proveedora de servicios de Internet (ISP) que brinda conectividad residencial y corporativa en la región. TeleConecta cuenta con un área de Soporte Técnico Nivel 1 encargada de recibir, diagnosticar y resolver (o escalar) los reclamos de conectividad reportados por sus clientes: cortes de servicio, lentitud de conexión, intermitencias, y problemas de configuración del módem/router.

#### El proceso elegido: Soporte Técnico Nivel 1

Actualmente, este proceso se gestiona de forma manual: el cliente llama o escribe a un canal de atención, un operador humano toma nota del reclamo, realiza preguntas básicas de diagnóstico (¿tiene luces el módem?, ¿reinició el equipo?, ¿el problema es general o en un dispositivo puntual?) y, según las respuestas, decide si puede resolverlo en el momento o si debe derivarlo a Nivel 2 (soporte técnico de campo o especializado).

#### Justificación: por qué este proceso es ineficiente hoy

Se identificaron las siguientes ineficiencias en el modelo actual:

- Alta demanda de consultas repetitivas: un porcentaje significativo de los reclamos corresponde a soluciones simples y estandarizables (reinicio de equipo, verificación de cableado, restablecimiento de configuración), que no requieren intervención humana especializada pero ocupan tiempo de los operadores.
- Tiempos de espera elevados: al depender de la disponibilidad de un operador, los clientes deben esperar en cola incluso para diagnósticos básicos.
- Falta de registro estandarizado: sin un sistema que centralice los datos del reclamo (tipo de problema, diagnóstico aplicado, resultado), se dificulta el seguimiento histórico y la generación de métricas.
- Inconsistencia en el diagnóstico: al no seguir un árbol de decisión formalizado, distintos operadores pueden aplicar distintos pasos para un mismo tipo de problema, afectando la calidad del servicio.

#### Propuesta de solución

Frente a este escenario, se propone automatizar el primer nivel de atención mediante un chatbot de soporte técnico, que simula la interacción con el cliente, aplique un árbol de diagnóstico estandarizado modelado en BPMN 2.0, registre cada caso en una base de datos (plantilla simulada), y derive a Nivel 2 únicamente los casos que efectivamente lo requieran. De esta forma, se libera tiempo del equipo humano, se reducen los tiempos de espera para diagnósticos simples, y se estandariza la calidad y trazabilidad del proceso.

Link a los diagramas en [draw.io](https://draw.io):

[https://drive.google.com/file/d/1PMz855ilXDU\\_J0esdOfaL9\\_DoUgY3lta/view?usp=drive\\_link](https://drive.google.com/file/d/1PMz855ilXDU_J0esdOfaL9_DoUgY3lta/view?usp=drive_link)

# 1- MODELADO DE NEGOCIO (BPMN)

## Flujo AS-IS (proceso manual actual)

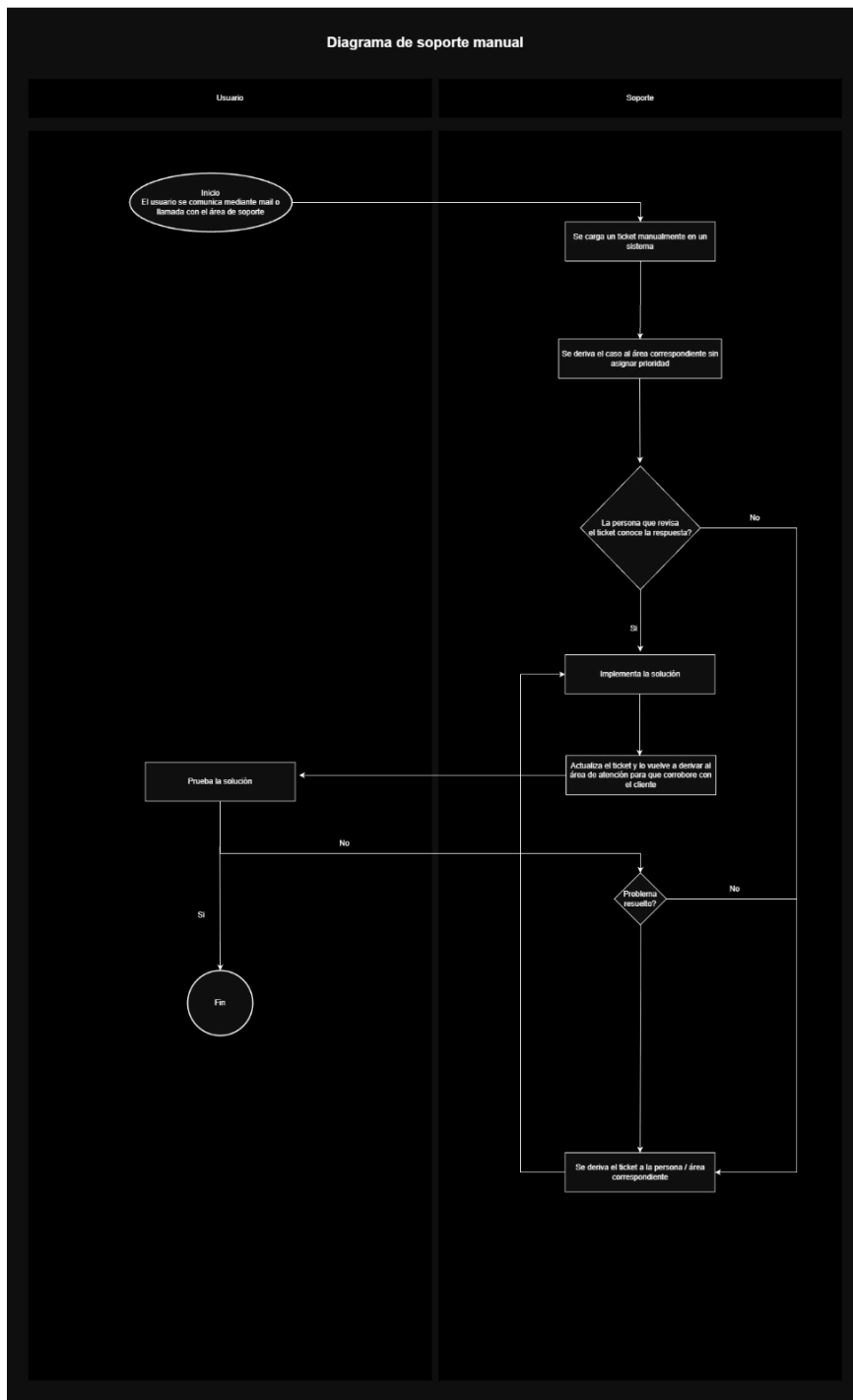


Figura 1: Diagrama BPMN del proceso de soporte técnico AS-IS (situación actual).

El proceso actual se inicia cuando el usuario se comunica con el área de soporte por mail o llamada. El operador de soporte carga el ticket manualmente en un sistema y lo deriva al área correspondiente sin asignar prioridad. Luego se evalúa si la persona que revisa el ticket conoce la respuesta: si no la conoce, el caso se redirige nuevamente hasta encontrar a alguien que pueda resolverlo, generando demoras. Si la conoce, implementa la solución, actualiza el ticket y lo deriva de vuelta al área de atención para que el usuario la pruebe. Si el problema no queda resuelto, el ticket vuelve a derivarse, repitiendo el ciclo hasta su cierre.

Este flujo evidencia las insuficiencias descritas en la introducción: ausencia de priorización, dependencia total del conocimiento del operador de turno, y múltiples idas y vueltas antes de llegar a una resolución.

### Flujo TO-BE (proceso optimizado con el chatbot)

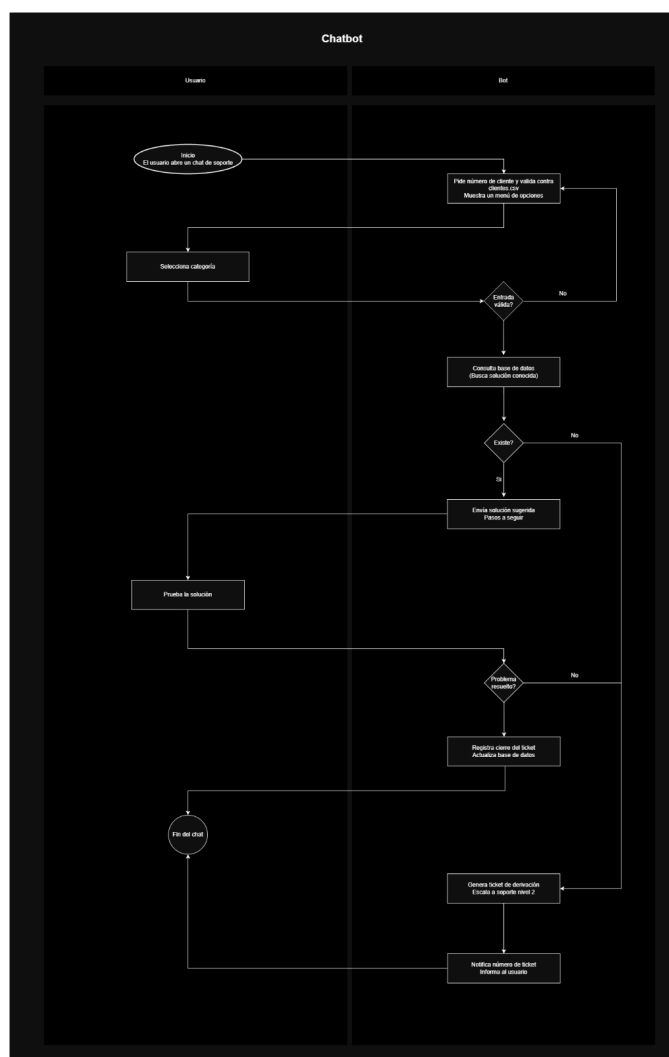


Figura 2: Diagrama BPMN del proceso de soporte técnico TO-BE (con chatbot de Nivel 1).

El proceso optimizado se representa mediante dos carriles (lanes): Usuario y Bot, que diferencian claramente las tareas de espera de entrada de datos de las tareas automáticas del sistema.

- **Evento de inicio:** el usuario abre un chat de soporte.

- **Tarea del Bot:** solicita el número de cliente y lo valida contra *clientes.csv*, mostrando luego un menú de opciones.
- **Tarea del Usuario:** selecciona una categoría de consulta del menú.
- **Gateway 1 — ¿Entrada válida?:** si la opción ingresada no es válida, el bot vuelve a mostrar el menú (bucle de corrección). Si es válida, continúa el flujo.
- **Tarea del Bot:** consulta la base de datos (*OPCIONES SOPORTE*) buscando una solución conocida para la categoría seleccionada.
- **Gateway 2 — ¿Existe solución?:** si no existe una solución registrada para esa categoría, el caso se deriva directamente a generación de ticket de escalamiento. Si existe, el bot envía la solución sugerida con los pasos a seguir.
- **Tarea del Usuario:** prueba la solución indicada.
- **Gateway 3 — ¿Problema resuelto?:** si el problema **no** se resolvió, el flujo deriva a la generación de un ticket de escalamiento a soporte Nivel 2. Si **sí** se resolvió, el bot registra el cierre del ticket y actualiza la base de datos.
- **Camino de escalamiento:** cuando el problema no pudo resolverse (por falta de solución conocida o porque la solución sugerida no funcionó), el bot genera un ticket de derivación, escalando el caso a soporte Nivel 2, y notifica al usuario el número de ticket generado.
- **Evento de fin:** fin del chat, ya sea por resolución autónoma o por derivación con ticket asignado.

Esta estandarización permite que el equipo humano de Nivel 2 reciba únicamente los casos que realmente requieren intervención especializada, reduciendo la carga operativa y los tiempos de respuesta para el cliente.

## 2- DICCIONARIO DE DATOS

El sistema gestiona tres estructuras de datos principales: dos persistidas en archivos CSV funcionan como base de datos simulada y una embebida en el código como diccionario en memoria.

Entidad: CLIENTES (*archivo clientes.csv*)

Base de datos simulada de clientes existentes, utilizada para validar el número de cliente al inicio de la conversación. Base de datos simulada de clientes existentes, utilizada para validar el número de cliente al inicio de la conversación.

| Campo          | Tipo de dato         | Descripción                                                                               | Ejemplo            |
|----------------|----------------------|-------------------------------------------------------------------------------------------|--------------------|
| numero_cliente | Texto (numérico, PK) | Identificador único del cliente. Se valida que exista en este archivo antes de continuar. | 100001             |
| nombre         | Texto                | Nombre completo del cliente.                                                              | Juan Perez         |
| plan           | Texto                | Plan de servicio contratado.                                                              | Internet 300 Megas |
| direccion      | Texto                | Dirección registrada del cliente.                                                         | Av. Rivadavia 1234 |

Tabla 1: Diccionario de datos de la entidad *CLIENTES*.

Entidad: TICKETS (*archivo tickets.csv*)

Registro de cada consulta o reclamo generado durante la conversación con el bot. Esta entidad constituye la evidencia de persistencia y trazabilidad del proceso: cada interacción relevante queda registrada con su categoría, descripción y nivel de atención asignado.

| Campo          | Tipo de dato                         | Descripción                                                                          | Ejemplo                                         |
|----------------|--------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------|
| id             | Texto (PK)                           | Identificador único del ticket, formato <i>TCK-XXX</i> incremental.                  | TCK-001                                         |
| numero_cliente | Texto (FK → clientes.numero_cliente) | Cliente que generó el ticket.                                                        | 100001                                          |
| nombre         | Texto                                | Nombre del cliente, copiado al momento de crear el ticket.                           | Juan Perez                                      |
| categoría      | Texto                                | Opción del menú elegida por el cliente.                                              | No tengo conexión a internet / se corta seguido |
| descripción    | Texto largo                          | Detalle adicional ingresado por el cliente (solo si no se resolvió o eligió "Otro"). | Se corta cada 10 minutos                        |
| nivel          | Texto (enum)                         | Nivel de atención asignado al ticket.                                                | Nivel 1 (autoservicio) / Nivel 2                |

Tabla 2: Diccionario de datos de la entidad *TICKETS*.

OPCIONES SOPORTE (*en memoria, no persiste en CSV*)

Diccionario embebido en el código fuente con las categorías de consulta disponibles y su solución sugerida. A diferencia de las dos entidades anteriores, esta estructura no se persiste en disco: se carga en memoria al iniciar el programa y es consultada por el bot durante el Gateway "¿Existe solución?" del flujo TO-BE.

| Campo | Tipo de dato | Descripción | Ejemplo |
|-------|--------------|-------------|---------|
|-------|--------------|-------------|---------|

|                 |                         |                                                          |                                          |
|-----------------|-------------------------|----------------------------------------------------------|------------------------------------------|
| clave           | Texto<br>(numérico, PK) | Número de opción del menú.                               | "1"                                      |
| descripció<br>n | Texto                   | Texto de la categoría mostrado<br>en el menú.            | Quiero cambiar la<br>contraseña del WiFi |
| solución        | Texto largo             | Pasos sugeridos por el bot para<br>resolver el problema. | "1. Ingresá a la Sucursal<br>Virtual..." |

Tabla 3: Diccionario de datos de la estructura OPCIONES\_SOPORTE.

### 3- ARQUITECTURA Y STACK TÉCNICO

#### Lenguaje de programación

Se seleccionó Python como lenguaje de desarrollo del chatbot, principalmente por dos motivos: en primer lugar, es el lenguaje trabajado a lo largo de la cursada en la materia de Programación 1, lo que permite aplicar directamente los conocimientos adquiridos. En segundo lugar, ofrece estructuras de datos nativas como diccionarios, listas, tuplas y módulos estándar para manejo de archivos CSV (**csv**) que resultan ideales para simular tanto la base de datos de clientes y tickets como el diccionario de soluciones embebido, sin necesidad de instalar dependencias externas.

#### Plataforma elegida: simulador en consola

Para la implementación se optó por un simulador en consola, en lugar de integrar el bot directamente con una plataforma de mensajería real como Telegram API o WhatsApp Business. Esta decisión responde al alcance académico y los objetivos pedagógicos del TP: priorizar la correcta implementación de la lógica de negocio tales como validaciones de entrada, máquina de estados, persistencia de datos y manejo del camino infeliz por sobre la complejidad técnica de integrar una API externa, que implicaría gestionar webhooks, tokens de autenticación y despliegue en un servidor accesible públicamente.

Cabe destacar que esta elección no compromete la validez del modelo BPMN ni la robustez de la solución: la lógica de estados, validaciones y persistencia desarrollada es independiente del canal de comunicación utilizado. Esto significa que el mismo código de lógica de negocio podría adaptarse a futuro para operar sobre Telegram API (reemplazando las funciones `input()` y `print()` por los métodos correspondientes de la librería `python-telegram-bot`) sin necesidad de rediseñar el proceso ni el modelo BPMN.

#### Persistencia de datos

El requisito de persistencia se cumple mediante el uso de archivos CSV que funcionan como base de datos simulada:

- `clientes.csv`: contiene el padrón de clientes existentes, utilizado para validar el número de cliente ingresado al inicio de cada conversación.
- `tickets.csv`: registra cada caso generado durante la interacción con el bot, incluyendo su categoría, descripción y nivel de atención asignado (Nivel 1 resuelto por autoservicio, o Nivel 2 escalado).

### 5- Máquina de Estados

### Estados identificados

El proceso comienza en el estado INICIO, donde el bot saluda al usuario y solicita su número de cliente, dentro de la función *chatbot()*. A partir de ahí, el sistema entra en VALIDANDO\_CLIENTE, gestionado por *pedir\_numero\_cliente()*: si el número ingresado está vacío, contiene caracteres no numéricos, o no existe en *clientes.csv*, el bot permanece en este mismo estado y vuelve a solicitar el dato, repitiendo el ciclo hasta recibir un número válido.

Una vez validado el cliente, se pasa a MOSTRANDO\_MENU, donde *mostrar\_menu()* presenta las cinco categorías de soporte disponibles, y de inmediato a ESPERANDO\_OPCION, dentro del bucle principal de *chatbot()*: si la opción ingresada no corresponde a ninguna de las disponibles (1 a 5), el bot permanece en este estado y vuelve a mostrar el menú. Si la opción elegida está entre 1 y 4, el flujo avanza a MOSTRANDO\_SOLUCION, donde se muestra al usuario la solución sugerida correspondiente a esa categoría. Inmediatamente después, el bot entra en ESPERANDO\_CONFIRMACION, gestionado por *pedir\_confirmacion()*, donde se pregunta si la solución resolvió el problema: cualquier respuesta distinta de "s" o "n" mantiene al bot en este mismo estado hasta recibir una respuesta válida.

A partir de la confirmación, el proceso se bifurca en dos caminos. Si la respuesta es afirmativa, se pasa a REGISTRANDO\_RESOLUCION: se genera un ticket con nivel "Nivel 1 (autoservicio)" y la conversación se cierra. Si la respuesta es negativa —o si el usuario eligió directamente la opción 5 del menú ("Ninguna de las anteriores")— el bot pasa a ESPERANDO\_DETALLE, solicitando una breve descripción del problema antes de continuar.

Desde ESPERANDO\_DETALLE, el flujo avanza siempre a ESCALANDO\_NIVEL2: se genera un ticket con nivel "Nivel 2", se persiste en *tickets.csv*, y se notifica al usuario el número de ticket asignado. Tanto este camino como el de REGISTRANDO\_RESOLUCION convergen finalmente en el estado FIN, que marca el cierre de la conversación.

### Persistencia del estado entre pasos

A diferencia de una aplicación web sin memoria, el bot mantiene el contexto de la conversación a través de las variables *cliente* (datos del cliente ya validado) y *tickets* (lista cargada en memoria desde *tickets.csv*), que se transportan entre los distintos estados durante toda la ejecución de *chatbot()*. Esto garantiza que, una vez identificado el cliente, el bot no vuelva a solicitar ese dato en ningún paso posterior del proceso.

### Limitaciones de la simulación

Al tratarse de una simulación en consola de una única sesión, no se contempla la persistencia del estado entre distintas ejecuciones del programa: si la conversación se interrumpe, no se retoma desde el punto donde quedó. En una migración a una plataforma real (Telegram, WhatsApp), esta limitación se resolvería agregando una tabla *sesiones.csv* que guarde el estado actual de cada usuario por su identificador de chat.

## **6- CAMINO INFELIZ Y PRUEBAS DE ESTRÉS**

El recorrido normal del proceso el camino feliz, donde el usuario ingresa siempre datos correctos y el bot avanza sin interrupciones hasta el cierre del ticket— ya fue descrito en la Sección 5 Máquina de Estados. A continuación se describe el camino infeliz: los puntos donde la entrada del usuario puede no ajustarse a lo esperado, y cómo el bot responde sin interrumpir la conversación. El sistema contempla los siguientes escenarios de error, informando el problema con un mensaje claro y volviendo a solicitar el dato sin avanzar de estado:



a- Número de cliente vacío o no numérico. Si el usuario no ingresa nada, o ingresa caracteres no numéricos (ej. "abc123"), el bot responde con el mensaje correspondiente ("no puede estar vacío" / "solo puede contener dígitos") y vuelve a pedir el dato.

b- Número de cliente inexistente: Si el formato es correcto pero el número no figura en *clientes.csv* (ej. "999999"), el bot informa que no encontró ese cliente y solicita verificarlo. Es un error de negocio, no de formato.

c- Opción de menú fuera de rango: Si el usuario ingresa una opción que no está entre las disponibles (ej. "9" o "hola"), el bot avisa que la opción es inválida y vuelve a mostrar el menú, sin perder el contexto del cliente ya identificado.

e- Confirmación con respuesta ambigua: Si al preguntar si la solución resolvió el problema el usuario responde algo distinto de "s" o "n" (ej. "tal vez"), el bot repite la pregunta hasta obtener una respuesta interpretable.

f- Detalle del problema sin completar: Si el usuario no ingresa ningún detalle al ser escalado a Nivel 2, el bot no bloquea el proceso: registra el ticket igual, con el valor por defecto "Sin detalle adicional."

En todos los casos, el manejo de errores se resuelve mediante validaciones explícitas y bucles *while True* con *continue*, evitando excepciones no controladas o finalizaciones abruptas. Esto demuestra que el camino infeliz es una característica transversal del diseño, presente en cada punto de entrada de datos del usuario.

## 7- MANUAL DE USUARIO

Al iniciar el bot, el sistema solicita el número de cliente. Este dato es obligatorio y debe coincidir con uno registrado previamente en la base de clientes. Si el usuario no lo recuerda o lo ingresa incorrectamente, el bot lo orienta con mensajes claros hasta que ingrese un número válido. Una vez identificado, el bot presenta un menú con cinco categorías de consulta, cada una accesible escribiendo el número correspondiente: 1) no tengo conexión a internet o se corta seguido, 2) quiero cambiar la contraseña del WiFi, 3) consulta o reclamo sobre mi factura, 4) quiero cambiar de plan, y 5) ninguna de las anteriores o no se resolvió mi problema. Para seleccionar una opción, el usuario debe escribir únicamente el número correspondiente (por ejemplo, "1") y presionar Enter.

Si elige una opción entre 1 y 4, el bot muestra una solución sugerida con los pasos a seguir. Luego de leerla, se le pregunta al usuario si esto resolvió su consulta, debiendo responder "s" (sí) o "n" (no). Si responde "s", la conversación finaliza y se confirma la resolución del caso. Si responde "n", se le solicita un breve detalle adicional y el caso se deriva automáticamente a un especialista de soporte Nivel 2. Si en cambio elige la opción 5 —porque ninguna categoría del menú representa su problema—, puede seleccionarla directamente: el bot le solicitará una breve descripción del inconveniente y derivará el caso a soporte Nivel 2 sin pasar por la etapa de confirmación.

En todos los casos en que el caso se deriva a Nivel 2, el bot informa un número de ticket con formato *TCK-XXX*, que el usuario debe conservar como comprobante de su reclamo. Para un uso fluido del bot, se recomienda al usuario ingresar el número de cliente exactamente como figura en la factura o en la Sucursal Virtual, responder únicamente con las opciones indicadas por el bot (números del menú, o "s"/"n" en las confirmaciones) para evitar reintentos innecesarios, y utilizar la opción 5 sin dudar en caso de no encontrar su consulta en el menú, ya que el sistema está diseñado para derivar cualquier caso no contemplado a un especialista humano.

## 8- Conclusión

El desarrollo de este trabajo integrador permitió comprender, en la práctica, el vínculo entre el modelado de procesos de negocio y la automatización mediante un chatbot. El verdadero desafío no

estuvo en la programación en sí, sino en pensar primero como consultores tecnológicos: identificar una ineficiencia real dentro del proyecto, entender por qué el proceso manual de Soporte Técnico Nivel 1 generaba demoras e inconsistencias, y recién después diseñar una solución coherente con esa lógica de negocio. El uso de BPMN 2.0 resultó clave en ese proceso de pensamiento. Diagramar primero el flujo AS-IS obligó a reflexionar sobre los puntos débiles del proceso actual —la falta de priorización, la dependencia del conocimiento de cada operador, las idas y vueltas innecesarias— antes de proponer cualquier mejora. A partir de ahí, el diseño del flujo TO-BE no fue una decisión arbitraria, sino la traducción directa de esas ineficiencias en decisiones concretas: los tres gateways del diagrama (validez de la entrada, existencia de una solución conocida, y resolución efectiva del problema) representan exactamente los tres momentos donde el proceso real necesitaba un criterio claro para avanzar o derivar el caso.

Esta forma de trabajar —pensar el "qué" antes que el "cómo"— también se reflejó en la lógica de la máquina de estados y en el tratamiento del camino infeliz. Antes de escribir código, fue necesario imaginar todos los caminos posibles que un usuario real podría tomar, incluyendo los errores, y decidir conscientemente qué debía hacer el bot en cada uno de esos casos para no interrumpir la experiencia ni perder información del proceso. Este trabajo demostró que automatizar un proceso administrativo no es simplemente escribir instrucciones en un lenguaje de programación, sino primero comprender a fondo cómo funciona ese proceso en la realidad, identificar sus puntos críticos de decisión, y solo entonces plasmar esa lógica en una herramienta que la respete fielmente.

## **Anexo**

### **IA UTILIZADA:**

En las siguientes imágenes se puede observar la interacción con la herramienta CLAUDE en donde se le pidió un esbozo de la estructura y esqueleto del Trabajo Integrador PDF. La cual sugiero el esquema que se puede ver en las capturas de pantallas realizadas. También se utilizó en otras ocasiones para dudas puntuales a la hora de la realización del presente proyecto.

P: ¿Qué tenés ya avanzado del trabajo?  
R: Tengo todo (BPMN + código) pero nada del PDF  
P: ¿Qué proceso administrativo elegiste para el chatbot?  
R: Soporte técnico nivel 1

Perfecto, eso simplifica todo — ya tenés la parte difícil hecha. Ahora es "solo" organizar y redactar.

Contame un poco más para armarte la estructura exacta del PDF:

P: ¿En qué plataforma/lenguaje hiciste el bot?  
R: Simulador en consola (sin plataforma real)  
P: ¿Cómo manejas la persistencia/base de datos?  
R: Excel/CSV como plantilla  
P: ¿Tenés capturas de las consultas a IA (Copilot/Gemini/Claude) guardadas?  
R: No tengo, hay que armarlas

Bien, ya tengo el panorama completo. Te armo la estructura del PDF según el enunciado (4-8 páginas, Times New Roman 11pt, títulos en 12pt BOLD MAYÚSCULAS).

## Estructura recomendada del PDF

### 1. PORTADA (no cuenta como página de contenido normalmente, pero la incluís)

- Título del TP, materia, cátedra, integrante(s), comisión, fecha.

### 2. INTRODUCCIÓN / FUNDAMENTO

- Organización elegida (real o ficticia) y breve contexto.
- Proceso elegido: **Soporte Técnico Nivel 1** — por qué es ineficiente/manual hoy (justificación de negocio).

### 3. MODELADO DE NEGOCIO (BPMN)

- Diagrama BPMN "AS-IS" (proceso actual, manual).
- Diagrama BPMN "TO-BE" (proceso optimizado con el bot).
- Explicación de carriles (Usuario / Sistema-Bot), eventos de inicio/fin, y al menos 2 gateways con sus condiciones (ej. ¿problema resuelto con FAQ?, ¿requiere escalamiento a Nivel 2?).
- *(Acá van tus imágenes del BPMN, centradas, con aclaración debajo)*

### 4. ARQUITECTURA Y STACK TÉCNICO

- Lenguaje: Python.
- Plataforma: simulador en consola (justificar por qué — practicidad para el alcance del TP, posibilidad de migrar a Telegram API más adelante).
- Persistencia: plantilla Excel/CSV — qué columnas tiene, qué registra.

## 5. DICCIONARIO DE DATOS

- Tabla con variables/entidades: nombre, tipo, descripción (ej. `id_ticket`, `usuario`, `estado`, `categoria_problema`, `fecha`, `resuelto`).

## 6. MÁQUINA DE ESTADOS

- Listado de estados posibles (ej. `INICIO`, `ESPERANDO_DESCRIPCION`, `BUSCANDO_SOLUCION`, `RESUELTO`, `ESCALADO`, `FIN`) y transiciones.

## 7. MANUAL DE USUARIO

- Comandos/opciones del bot, ejemplos de interacción.

## 8. CAMINO INFELIZ / PRUEBAS DE ESTRÉS

- Ejemplos concretos: usuario ingresa texto donde se espera número, opción inválida, etc., y cómo responde el bot.

## 9. USO DE HERRAMIENTAS DE IA

- Qué herramientas usaste y para qué (diseño BPMN, lógica del código, debugging).
- Capturas de prompts y respuestas — **esto falta armarlo**, te ayudo abajo.

## 10. CONCLUSIONES

- Breve cierre: qué aprendiste, qué integrarías a futuro (DB real, Telegram, etc).